Chapter 8 — Individual Assignment Report

Employee Directory with Vue 3, Axios, Express and MySQL

Name: Mohammed Al-Sakkaf **Matric No: Course:** Web Application Development — Chapter 8

---

1. Working UI — Screenshots

> *Replace each box below with a real screenshot of your running app (http://localhost:5174). Take them after running sql/schema.sql and starting both servers.*

Screenshot 1 — Employee list with seeded data and active headcount `[ INSERT SCREENSHOT: full page showing the table of 7 employees + the "Active employees" count ]`

Screenshot 2 — Add/Edit form with inline validation errors `[ INSERT SCREENSHOT: submit the form with a bad Employee ID and salary so the red error messages show ]`

Screenshot 3 — Server-side search and sort `[ INSERT SCREENSHOT: type a name in Search and choose Sort by Salary (descending) ]`

Screenshot 4 — DevTools Network tab during a CRUD action `[ INSERT SCREENSHOT: Network tab showing a request to localhost:3001 (e.g. POST /employees) ]`

2. How the code satisfies each Learning Outcome

LO1 — Connecting Vue to a backend. The Vue app communicates with a Node.js + Express REST API (server/index.js) that I wrote, which persists data in MySQL through the mysql2 Promise pool (server/db.js). The front-end never touches the database directly; the Axios baseURL (http://localhost:3001) points at the API. On page load, App.vue's onMounted(load) fetches all employees via GET /employees.

LO2 — Axios HTTP client. All HTTP traffic goes through a single reusable Axios instance in src/api/employeeApi.js (configured baseURL, timeout, default headers). A request interceptor logs the method and URL of every call; a response interceptor maps any server or network error into one human-readable userMessage and, for HTTP 400 validation failures, exposes field-level errors to the form. Components import named helpers (getEmployees, createEmployee, …) and never call axios directly. Every call uses async/await.

LO3 — Form handling in Vue. EmployeeForm.vue captures input with v-model, using .trim on text fields and .number on salary. A validate() function enforces all required rules (Employee ID regex ^EMP[0-9]{3,5}$, name length, email pattern, department, position, hire date not in the future, salary RM 1,500–50,000). Invalid fields render inline red error spans and the form refuses to submit while invalid. Valid data is emitted to the parent via a save event — children communicate only through props and emits.

LO4 — Persistent CRUD with search and sort. server/index.js implements the full CRUD lifecycle against MySQL using prepared statements (? placeholders) — Create, Read (list + single), Update and Delete. Server-side search uses SQL LIKE across four columns (name, empId, email, department) via ?q=. Server-side sort uses ORDER BY on a whitelisted set of columns (name, hireDate, salary, empId) via ?sortBy=&order=, so the client can never inject an arbitrary column name. All changes persist — restarting everything keeps the data.

3. Challenges and how I resolved them

- **MySQL returns DECIMAL (salary) and DATE as strings.** I normalised them on display — Number(salary) before currency formatting and slicing the date to YYYY-MM-DD so the date input binds correctly when editing.
- **Trusting client validation only is unsafe.** I duplicated the validation rules on the server and return HTTP 400 with field-level messages, then surfaced them through the Axios response interceptor back onto the form.
- **Editing the unique empId.** Allowing edits to the unique key caused duplicate-key conflicts, so I disable the Employee ID field in edit mode and handle ER_DUP_ENTRY with a 409 response.
- **CORS between ports 5174 and 3001.** Resolved with the cors() middleware on the Express app.

4. Sources consulted

Official documentation for Vue 3, Vite, Axios, Express and mysql2, and the Chapter 8 lab handout. All application code is my own.